

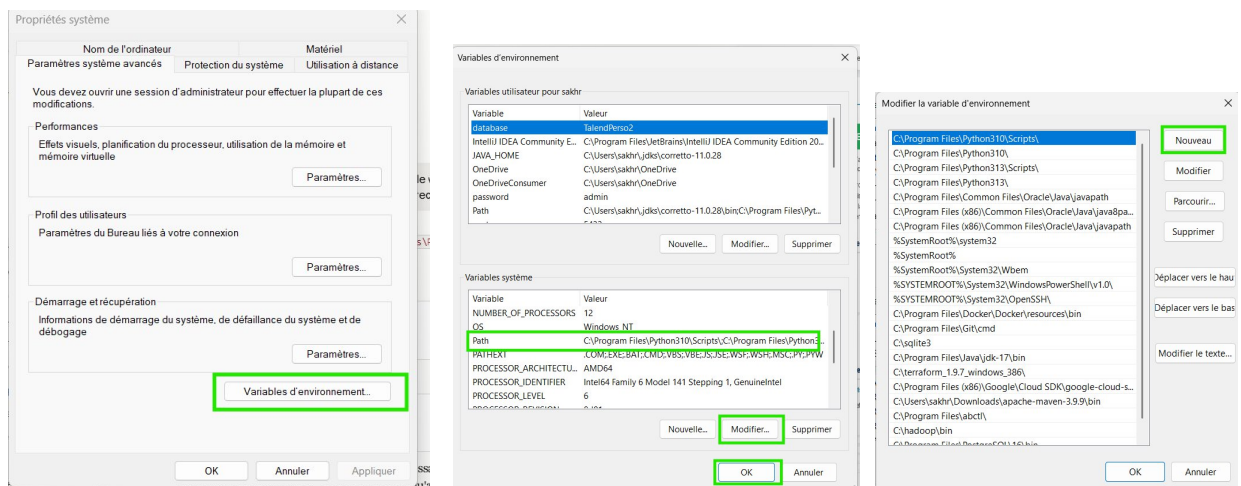
SUPPORT DE COURS - SQL

Ressources : <https://sql.sh/>

Prérequis : Installation PostgreSQL

!!! AJOUTER POSTGRES AUX VARIABLES D'ENVIRONNEMENT !!!

- Menu démarrer : variables d'environnement
- Modifier les variables d'environnement système
- Variables système => Path => Modifier => Nouveau => Ajouter



C:\Program Files\PostgreSQL\16\bin

Test :

```
>psql --version
psql (PostgreSQL) 16.4

>pg_dump --version
pg_dump (PostgreSQL) 16.4
```

Tous les exercices s'appuient sur un dataset e-commerce classique compose de 4 tables.

clients	commandes	ligne_commandes	produits
_client_id (PK) _nom _email _ville _pays _date_inscription	_client_id (FK) _commandes_id (PK) _date_commande _statut _total	_ligne_id (PK) _commande_id (FK) _produit_id(FK) _quantite _prix_unitaire	_produit_id (PK) _nom _categorie _prix _stock

Création des tables :

Syntaxe de base

```
CREATE TABLE nom_table (  
  col_1 TYPE CONTRAINTE,  
  col_2 TYPE CONTRAINTE,  
  ...  
)
```

Exemple de contraintes possibles :

```
CREATE TABLE nom_table (  
  col_1 TYPE PRIMARY KEY,           -- clé primaire  
  col_2 TYPE NOT NULL,              -- valeur obligatoire  
  col_3 TYPE UNIQUE,                -- valeur unique  
  col_4 TYPE DEFAULT valeur,        -- valeur par défaut  
  col_5 TYPE REFERENCES autre_table(col_autre_table) -- clé étrangère  
)
```

Exemple dans notre exercice :

```
CREATE TABLE nom_table (
  id          SERIAL          PRIMARY KEY,
  nom         VARCHAR(100) NOT NULL,
  email       VARCHAR(100) UNIQUE?,
  statut      VARCHAR(50)  DEFAULT 'actif',
  parent_id   INTEGER       REFERENCES autre_table(id)
)
```

Insertion de données dans des tables :

```
INSERT INTO nom_table (col_1, col_2, col_3)
VALUES
  (val1, val2, val3),
  (val4, val5, val6),
  (val7, val8, val9);
```

Modification d'une table (Quelques commandes) :

```
-- Ajouter une colonne
ALTER TABLE nom_table
ADD COLUMN nouvelle_col TYPE;
-- exemple
ALTER TABLE clients
ADD COLUMN email VARCHAR(100);
-- =====

-- Supprimer une colonne
ALTER TABLE nom_table
DROP COLUMN col_a_supprimer;
-- exemple :
ALTER TABLE clients
DROP COLUMN age;
-- =====
```

```

-- Modifier le type d'une colonne
ALTER TABLE nom_table
ALTER COLUMN col TYPE NOUVEAU_TYPE;
-- exemple :
ALTER TABLE clients
ALTER COLUMN nom TYPE VARCHAR(100);
-- =====

-- Renommer une colonne
ALTER TABLE nom_table
RENAME COLUMN ancien_nom TO nouveau_nom;
-- exemple :
ALTER TABLE clients
RENAME COLUMN nom TO nom_complet;
-- =====

-- Renommer la table
ALTER TABLE ancien_nom
RENAME TO nouveau_nom;
-- exemple :
ALTER TABLE clients
RENAME TO utilisateurs;
-- =====

-- Ajouter une contrainte
ALTER TABLE nom_table
ADD CONSTRAINT nom_contrainte CHECK (prix > 0);
-- exemple :
ALTER TABLE employes
ADD CONSTRAINT chk_salaire
CHECK (salaire > 0);
-- =====

-- Supprimer une contrainte
ALTER TABLE nom_table
DROP CONSTRAINT nom_contrainte;
-- exemple :
ALTER TABLE employes
DROP CONSTRAINT chk_salaire;
-- =====

-- Ajouter une clé étrangère après coup
ALTER TABLE nom_table
ADD CONSTRAINT nom_fk
FOREIGN KEY (col) REFERENCES autre_table(col);
-- exemple :
ALTER TABLE employes
ADD CONSTRAINT fk_departement
FOREIGN KEY (departement_id)
REFERENCES departements(id);
-- =====

```

```

-- Modifier une valeur
UPDATE nom_table
SET col_1 = nouvelle_valeur
WHERE condition;
-- exemple :
UPDATE employes
SET salaire = 2500
WHERE id = 3;
-- =====

-- Modifier plusieurs colonnes
UPDATE nom_table
SET col_1 = val1,
    col_2 = val2
WHERE condition;
-- exemple :
UPDATE employes
SET nom = 'Ali',
    salaire = 3000
WHERE id = 3;
-- =====

-- Supprimer des lignes
DELETE FROM nom_table
WHERE condition;
-- exemple :
DELETE FROM employes
WHERE salaire < 1000;
-- =====

-- Supprimer toutes les lignes (vider la table)
TRUNCATE TABLE nom_table;
-- exemple :
TRUNCATE TABLE commandes;
-- =====

-- Supprimer la table complètement
DROP TABLE nom_table;
DROP TABLE IF EXISTS nom_table; -- sans erreur si elle n'existe pas
-- exemple :
DROP TABLE commandes;
DROP TABLE IF EXISTS commandes;

```

CHAP 1 : SELECT / WHERE / ORDER BY / LIMIT

1.1 La requête SELECT

SELECT est la base de tout. Il permet de lire des données depuis une table. La structure minimale est :

```
SELECT colonne
FROM table;

-- Selectionner toutes les colonnes
SELECT *
FROM clients;
-- Selectionner des colonnes specifiques
SELECT nom, email, ville
FROM clients;
-- Renommer une colonne dans le resultat (alias)
SELECT nom AS client_nom, email AS adresse_email
FROM clients;
-- Calculer une valeur
SELECT nom, prix, prix * 1.20 AS prix_ttc
FROM produits;
```

1.2 Filtrer avec WHERE

WHERE permet de filtrer les lignes d'une table selon une ou plusieurs conditions. Les principaux opérateurs utilisables sont :

=, !=, <, >, <=, >=, LIKE, ILIKE, IN, BETWEEN, IS NULL, IS NOT NULL, AND, OR, NOT

```
-- Egalite
SELECT * FROM clients
WHERE pays = 'France';

-- Plusieurs conditions (AND / OR)
SELECT * FROM produits
WHERE categorie = 'Electronique' AND prix < 100;
```



```
-- Liste de valeurs (IN)
SELECT * FROM commandes
WHERE statut IN ('livre', 'expedie');

-- Intervalle (BETWEEN)
SELECT * FROM produits
WHERE prix BETWEEN 10 AND 50;

-- Recherche partielle (LIKE)
SELECT * FROM clients
WHERE email LIKE '%gmail.com'; -- se termine par gmail.com

SELECT * FROM produits
WHERE nom LIKE 'Cable%'; -- commence par Cable

SELECT * FROM produits
WHERE nom ILIKE '%usb%'; -- insensible a la casse

-- Valeurs nulles
SELECT * FROM clients
WHERE ville IS NULL;

-- Valeurs non nulles
SELECT * FROM
clients WHERE ville IS NOT NULL;
```

1.3 Trier avec ORDER BY et limiter avec LIMIT

```
-- Tri croissant (par défaut en ASC)
SELECT * FROM produits
ORDER BY prix ASC;
-- =====

-- Tri décroissant
SELECT * FROM produits
ORDER BY prix DESC;
-- =====

-- Tri sur plusieurs colonnes
SELECT * FROM produits
ORDER BY categorie ASC, prix DESC;
-- =====

-- limite de nombre de résultats
SELECT * FROM produits
ORDER BY prix DESC
LIMIT 5;
-- =====

-- OFFSET : sauter les N premiers résultats
SELECT * FROM produits
ORDER BY produit_id
LIMIT 10
OFFSET 20;

-- valeurs distinctes
SELECT DISTINCT categorie FROM produits
ORDER BY categorie
```


Exercices — Chapitre 1

Q1	[N1]	Afficher le nom et l'email de tous les clients.
Q2	[N1]	Afficher tous les produits de la categorie 'Electronique'.
Q3	[N1]	Afficher les produits dont le prix est superieur a 50 euros.
Q4	[N1]	Afficher les 5 produits les plus chers.
Q5	[N1]	Afficher les clients qui habitent a Paris ou Lyon.
Q6	[N1]	Afficher les commandes avec le statut 'livre' ou 'expedie'.
Q7	[N1]	Afficher les produits dont le nom contient le mot 'cable' (insensible a la casse).
Q8	[N1]	Afficher les clients sans ville renseignee.
Q9	[N1]	Afficher les produits dont le prix est entre 20 et 100 euros, tries par prix croissant.
Q10	[N1]	Afficher la liste des categories distinctes de produits.
Q11	[N1]	Afficher les commandes passees en 2023, tries du plus recent au plus ancien.
		<i>Indice : Utilisez BETWEEN avec des dates ou >= et <=</i>
Q12	[N1]	Afficher le nom et le prix TTC (prix * 1.20) de tous les produits, avec l'alias prix_ttc.
Q13	[N1]	Afficher les 3 produits avec le moins de stock.
Q14	[N1]	Afficher les clients inscrits apres le 1er janvier 2022.
Q15	[N1]	Afficher les commandes dont le total est superieur a 200 euros et le statut 'livre'.

CHAP 2 : FONCTION D'AGGREGATION, GROUP BY, HAVING.

2.1 Fonctions d'agrégation

Les fonctions d'agrégation calculent une valeur à partir d'un ensemble de lignes. Elles s'utilisent avec GROUP BY, ou pour obtenir des valeurs globales

```
-- COUNT : compter les lignes
SELECT COUNT(*) AS nb_clients
FROM clients;
-- compter les lignes uniques
COUNT(DISTINCT ville) AS nb_villes
FROM clients;
-- SUM : somme
SELECT SUM(total) AS ca_total
FROM commandes;
-- AVG : moyenne
SELECT AVG(prix) AS prix_moyen
FROM produits;
-- MIN / MAX
SELECT MIN(prix) AS prix_min, MAX(prix) AS prix_max
FROM produits;
-- ROUND : arrondir
SELECT ROUND(AVG(prix)::NUMERIC, 2) AS prix_moyen
FROM produits;
```

2.2 GROUP BY — Grouper les résultats

La commande GROUP BY est utilisée en SQL pour grouper plusieurs résultats et utiliser une fonction de totaux sur un groupe de résultat. Sur une table qui contient toutes les ventes d'un magasin, il est par exemple possible de lister les ventes regroupées par clients identiques et d'obtenir le coût total des achats pour chaque client.

Toutes les colonnes affichées dans le SELECT qui ne sont pas utilisées dans une fonction d'agrégation doivent également être présentes dans la clause GROUP BY

```

-- Nombre de produits par categorie
SELECT categorie, COUNT(*) AS nb_produits
FROM produits
GROUP BY categorie
ORDER BY nb_produits DESC;

-- CA total par statut de commande SELECT statut,
SUM(total) AS ca_total
FROM commandes
GROUP BY statut;

-- Plusieurs colonnes de groupement
SELECT pays, ville, COUNT(*) AS nb_clients
FROM clients
GROUP BY pays, ville
ORDER BY pays, nb_clients DESC;

```

2.3 HAVING — Filtrer après agrégation

HAVING permet de filtrer les groupes créés par GROUP BY. Contrairement à WHERE, qui filtre les lignes avant le regroupement, HAVING filtre les résultats après le regroupement. Il est souvent utilisé avec des fonctions d'agrégation comme COUNT(), SUM(), AVG(), MIN() ou MAX().

Différence entre WHERE et HAVING

- WHERE : filtre les lignes avant le GROUP BY.
- HAVING : filtre les groupes après le GROUP BY.

```

-- Categories avec plus de 5 produits SELECT
categorie, COUNT(*) AS nb
FROM produits GROUP BY
categorie HAVING
COUNT(*) > 5;

-- Villes avec plus de 3 clients ET un email gmail
SELECT ville, COUNT(*) AS nb
FROM clients
WHERE email LIKE '%gmail%' -- filtre avant agregation GROUP BY ville
HAVING COUNT(*) > 3; -- filtre apres agregation

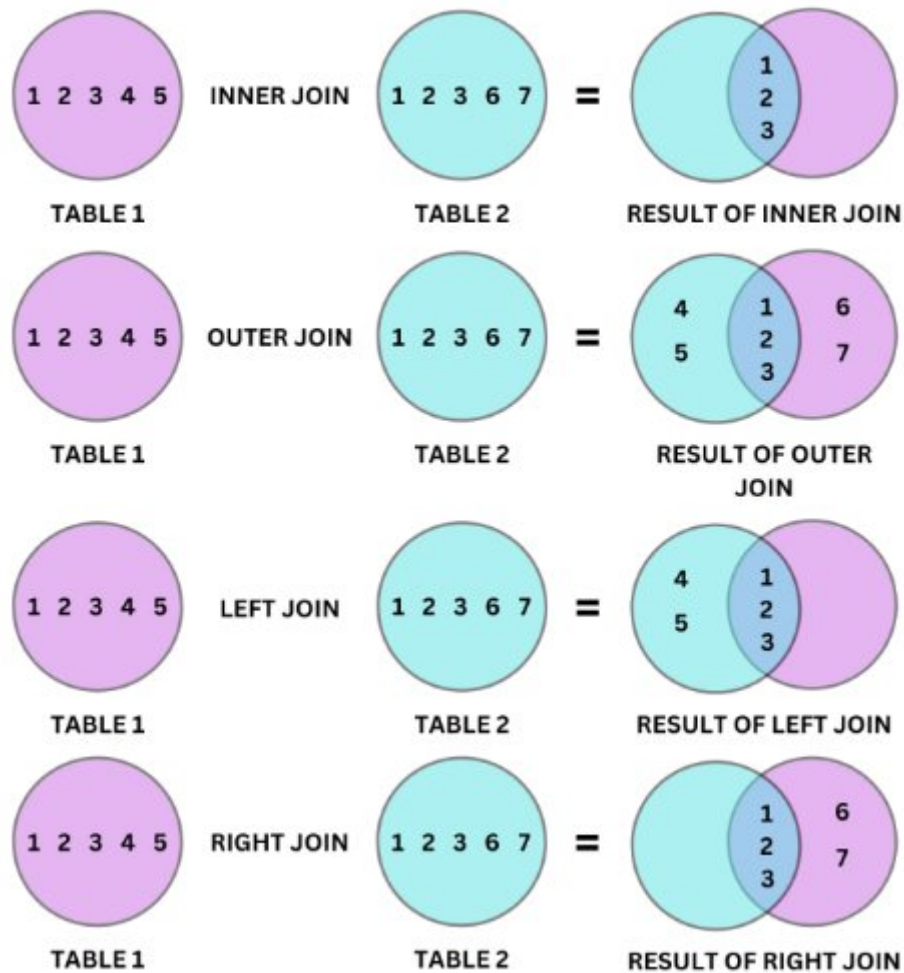
```

Exercices — Chapitre 2

Q16	[N1]	Compter le nombre total de clients.
Q17	[N1]	Calculer le prix moyen des produits.
Q18	[N1]	Afficher le produit le plus cher et le moins cher.
Q19	[N1]	Calculer le chiffre d'affaires total (somme des totaux de toutes les commandes).
Q20	[N1]	Compter le nombre de produits par categorie.
Q21	[N1]	Afficher le total des commandes par statut.
Q22	[N2]	Calculer le panier moyen (total moyen d'une commande) par mois.
		<i>Indice : Utilisez DATE_TRUNC('month', date_commande)</i>
Q23	[N2]	Afficher les categories avec un prix moyen superieur a 50 euros.
Q24	[N2]	Afficher le nombre de commandes par client (client_id + nombre de commandes).
Q25	[N2]	Afficher les clients ayant passe plus de 3 commandes.
Q26	[N2]	Afficher le CA total par mois, trie chronologiquement.
Q27	[N2]	Afficher les categories ayant au moins 3 produits en stock superieur a 0.
Q28	[N2]	Calculer la valeur totale du stock pour chaque categorie (prix * stock).
Q29	[N2]	Afficher le nombre de clients par pays, uniquement les pays avec plus de 10 clients.
Q30	[N2]	Afficher le mois ayant genere le plus de commandes.

CHAP 3 : LES JOINTURES

Type	Description	Lignes retournees
INNER JOIN	Intersection	Seulement les lignes qui ont une correspondance dans les 2 tables
LEFT JOIN	Gauche + intersection	Toutes les lignes de la table gauche + correspondances de droite (NULL si pas de match)
RIGHT JOIN	Droite + intersection	Toutes les lignes de la table droite + correspondances de gauche (NULL si pas de match)
FULL JOIN	Union	Toutes les lignes des 2 tables (NULL des 2 cotes si pas de match)



Exemple avec des tables :

Client (Cli)			Commandes (Cmd)			
	id_client [PK] integer	nom character varying (50)		id_commande [PK] integer	id_client integer	produit character varying (50)
1	1	Alice	1	101	1	Clavier
2	2	Bob	2	102	1	Souris
3	3	Charlie	3	103	2	Écran
4	4	David	4	104	5	Imprimante

Inner join :

```
SELECT c.*, co.*
FROM cli c
INNER JOIN cmd co
ON co.id_client = c.id_client;
```

	id_client integer	nom character varying (50)	id_commande integer	id_client integer	produit character varying (50)
1	1	Alice	101	1	Clavier
2	1	Alice	102	1	Souris
3	2	Bob	103	2	Écran

Full join :

```
SELECT c.*, co.*
FROM cli c
FULL JOIN cmd co
ON co.id_client = c.id_client;
```

	id_client integer	nom character varying (50)	id_commande integer	id_client integer	produit character varying (50)
1	1	Alice	101	1	Clavier
2	1	Alice	102	1	Souris
3	2	Bob	103	2	Écran
4	[null]	[null]	104	5	Imprimante
5	4	David	[null]	[null]	[null]
6	3	Charlie	[null]	[null]	[null]

Left join :

```
SELECT c.*, co.*
FROM cli c
LEFT JOIN cmd co
ON co.id_client = c.id_client;
```


	id_client integer 🔒	nom character varying (50) 🔒	id_commande integer 🔒	id_client integer 🔒	produit character varying (50) 🔒
1	1	Alice	101	1	Clavier
2	1	Alice	102	1	Souris
3	2	Bob	103	2	Écran
4	4	David	[null]	[null]	[null]
5	3	Charlie	[null]	[null]	[null]

Right join :

```
SELECT c.*, co.*
FROM cli c
RIGHT JOIN cmd co
ON co.id_client = c.id_client;
```

	id_client integer 🔒	nom character varying (50) 🔒	id_commande integer 🔒	id_client integer 🔒	produit character varying (50) 🔒
1	1	Alice	101	1	Clavier
2	1	Alice	102	1	Souris
3	2	Bob	103	2	Écran
4	[null]	[null]	104	5	Imprimante

Exercices — Chapitre 3

Q31	[N1]	Afficher toutes les commandes avec le nom et l'email du client associé.
Q32	[N1]	Afficher les lignes de commande avec le nom du produit et la quantité commandée.
Q33	[N2]	Afficher les clients qui n'ont jamais passé de commande.
		Indice : <i>LEFT JOIN + WHERE commande_id IS NULL</i>
Q34	[N2]	Afficher le détail complet des commandes : client, produit, quantité, sous-total.

Q35	[N2]	Afficher le CA total genere par chaque client (nom + total CA), trie par CA decroissant.
Q36	[N2]	Afficher le nombre de fois que chaque produit a ete commande.
Q37	[N2]	Afficher les produits qui n'ont jamais ete commandes.
		Indice : <i>LEFT JOIN + WHERE commande_id IS NULL</i>
Q38	[N2]	Afficher le CA total par categorie de produit.
Q39	[N2]	Afficher les 5 clients ayant le CA total le plus eleve.
Q40	[N2]	Afficher le produit le plus vendu (en quantite totale).
Q41	[N2]	Afficher pour chaque commande : nom client, nombre d'articles, total commande.
Q42	[N2]	Afficher les clients ayant commande des produits de la categorie 'Electronique'.
		Indice : <i>DISTINCT pour eviter les doublons</i>
Q43	[N2]	Afficher le CA moyen par commande pour chaque client.
Q44	[N2]	Afficher les produits vendus en 2023 avec leur categorie et quantite totale.
Q45	[N2]	Afficher les clients et le montant total de leurs commandes livrees uniquement.